

Contexto, Objetivo e Hipóteses do Projeto

- Contexto: . O volume de tráfego em rodovias metropolitanas é influenciado por diversos fatores, incluindo condições climáticas, horários do dia, dias da semana e padrões históricos de circulação. Compreender essas relações é fundamental para apoiar o planejamento urbano, a gestão da mobilidade e a previsão de demanda em sistemas de transporte. Neste projeto, utiliza-se uma série temporal de tráfego da rodovia I-94 com o objetivo de modelar e prever o volume horário de veículos. - Objetivo: . Desenvolver modelos de aprendizado de máquina capazes de prever o volume horário de tráfego a partir de variáveis climáticas, temporais e informações históricas da própria série. Além de buscar alta precisão preditiva, o projeto também visa identificar os fatores mais relevantes para a previsão e adotar uma metodologia adequada para dados temporais, minimizando riscos de vazamento de dados. - Hipóteses: . O volume de tráfego apresenta forte dependência temporal, sendo influenciado pelos volumes observados em períodos anteriores. Padrões cíclicos relacionados ao horário do dia e ao dia da semana possuem impacto significativo no comportamento do tráfego; . Condições climáticas, como temperatura, chuva e cobertura de nuvens, afetam o fluxo de veículos. Finais de semana e feriados apresentam padrões de tráfego distintos em relação aos dias úteis; . Modelos não lineares baseados em árvores de decisão tendem a capturar melhor a dinâmica do tráfego do que modelos lineares tradicionais.

Carregamento de Dados e Entendimento Inicial dos Dados

- O dataset utilizado contém registros horários de volume de tráfego da Interstate 94 Westbound, em Minnesota, além de informações climáticas e indicadores de feriados. Os dados abrangem aproximadamente seis anos de observações, permitindo analisar padrões diários, semanais e sazonais do tráfego. - A variável alvo do estudo é `traffic_volume`, enquanto as demais variáveis descrevem condições meteorológicas e características temporais associadas a cada observação.

- Carregamento das bibliotecas e dados

```
In [2]: # =====  
# Reprodutibilidade global (SEEDS)  
# =====  
  
import os  
import random  
import numpy as np  
  
SEED = 42  
  
# Garante determinismo em hashing interno do Python  
os.environ["PYTHONHASHSEED"] = str(SEED)  
  
# Seeds globais  
random.seed(SEED)  
np.random.seed(SEED)  
  
# =====  
# Básico e utilidades  
# =====  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
# =====
```

```

# Pré-processamento
# =====
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import RandomizedSearchCV, TimeSeriesSplit

# =====
# Modelos de regressão
# =====
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor

# =====
# Métricas de avaliação e +
# =====
import shap
from scipy.stats import randint, uniform
from scipy.stats import chi2_contingency
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, mean

# =====
# Diagnósticos estatísticos (opcional)
# =====
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox, het_breuschpagan
from statsmodels.tsa.stattools import adfuller, kpss
from arch.unitroot import PhillipsPerron
from scipy import stats
from scipy.stats import jarque_bera

# =====
# Dataset UCI
# =====
from ucimlrepo import fetch_ucirepo

# =====
# Warnings
# =====
import warnings
warnings.filterwarnings('ignore')

```

```

In [3]: # Dataset
metro_interstate_traffic_volume = fetch_ucirepo(id=492)

X = metro_interstate_traffic_volume.data.features
y = metro_interstate_traffic_volume.data.targets
df = pd.concat([X,y], axis = 1)
df.head()

```

```
Out[3]:
```

	holiday	temp	rain_1h	snow_1h	clouds_all	weather_main	weather_description	date_
0	NaN	288.28	0.0	0.0	40	Clouds	scattered clouds	2012-09-01 09:00
1	NaN	289.36	0.0	0.0	75	Clouds	broken clouds	2012-10-01 10:00
2	NaN	289.58	0.0	0.0	90	Clouds	overcast clouds	2012-11-01 11:00
3	NaN	290.13	0.0	0.0	90	Clouds	overcast clouds	2012-12-01 12:00
4	NaN	291.14	0.0	0.0	75	Clouds	broken clouds	2012-13-01 13:00

- Entendimento inicial

```
In [4]: linhas, colunas = df.shape
print(f'linhas: {linhas}, colunas: {colunas}')
```

linhas: 48204, colunas: 9

```
In [5]: # Valores faltantes
df.isna().sum()
```

```
Out[5]: holiday          48143
temp                    0
rain_1h                 0
snow_1h                 0
clouds_all              0
weather_main            0
weather_description     0
date_time               0
traffic_volume          0
dtype: int64
```

```
In [6]: print(f'Valores duplicados: {df.duplicated().sum()}')
```

Valores duplicados: 17

```
In [7]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48204 entries, 0 to 48203
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   holiday                61 non-null    object
1   temp                   48204 non-null float64
2   rain_1h                48204 non-null float64
3   snow_1h                48204 non-null float64
4   clouds_all             48204 non-null int64
5   weather_main           48204 non-null object
6   weather_description    48204 non-null object
7   date_time              48204 non-null object
8   traffic_volume         48204 non-null int64
dtypes: float64(3), int64(2), object(4)
memory usage: 3.3+ MB
```

- Identificação prévia de potenciais problemas

- Data leakage: . Depende do tratamento de date_time. - Tipos errados (datas como string) . Será corrigido com a criação de colunas de dia e semana aliado a transformação seno e cosseno. - Variáveis com significado especial: . "temp", "rain_1h", "snow_1h", "clouds_all" e colunas que serão derivadas de date_time são variáveis contínuas. Então deve-se usar apenas o StandarScaler; . "traffic_volume" é a variável target e é altamente correlacionado (Time series) - Variáveis derivadas do target: . Não há Função direta de traffic_volume ou construída a partir de dados futuros ou um componente matemático do target.

Análise Exploratória

- Dataset temporário

```
In [8]: df_eda = df.copy()
```

```
In [9]: # Dropando as 17 linhas duplicadas
df_eda = df_eda.drop_duplicates().reset_index(drop=True)
```

```
In [10]: # NaN para este dataset = dia normal
# Feriado = 1; dia normal = 0
df_eda["is_holiday"] = df_eda["holiday"].notna().astype(int)
df_eda.drop(columns = ["holiday"], inplace = True)

# tráfego urbano é altamente periódico
# há forte sazonalidade diária e semanal
df_eda["date_time"] = pd.to_datetime(df_eda["date_time"])

# Extrair hora e dia da semana
df_eda["hour"] = df_eda["date_time"].dt.hour
df_eda["weekday"] = df_eda["date_time"].dt.weekday

# Aplicar seno/cosseno
# Poderíamos ir além: harmônico de 12h → bem fundamentado, dado o padrão oscilatório
df_eda["hour_sin"] = np.sin(2 * np.pi * df_eda["hour"] / 24)
df_eda["hour_cos"] = np.cos(2 * np.pi * df_eda["hour"] / 24)
```

```

df_eda["weekday_sin"] = np.sin(2 * np.pi * df_eda["weekday"] / 7)
df_eda["weekday_cos"] = np.cos(2 * np.pi * df_eda["weekday"] / 7)

# É final de semana = 1; dia de semana = 0
df_eda["is_weekend"] = df_eda["weekday"].isin([5,6]).astype(int)

# Remover colunas intermediárias
df_eda = df_eda.drop(columns=["date_time", "hour", "weekday"])

```

- Classificação das variáveis do dataset temporário

```

In [11]: cols_bin = ["is_holiday", "is_weekend"]

cols_cat = ["weather_main", "weather_description"]

cols_num = ["temp", "rain_1h", "snow_1h", "clouds_all", "hour_sin", "hour_cos", "we

cols_time = ["date_time", "hour", "weekday"]

target = "traffic_volume"

```

- Análise temporal

```

In [12]: # Funções de Teste de Estacionariedade
def adf_test(series, alpha=0.05):
    result = adfuller(series, autolag="AIC", regression="c")
    stat, pvalue = result[0], result[1]

    print("=== ADF (Augmented Dickey-Fuller) ===")
    print(f"ADF Statistic : {stat:.4f}")
    print(f"p-value       : {pvalue:.4f}")

    if pvalue < alpha:
        print("➡ Conclusão: Série ESTACIONÁRIA (rejeita H0 de raiz unitária)\n")
    else:
        print("➡ Conclusão: Série NÃO estacionária (não rejeita H0)\n")

def kpss_test(series, alpha=0.05):
    result = kpss(series, regression="c", nlags="auto")
    stat, pvalue = result[0], result[1]

    print("=== KPSS ===")
    print(f"KPSS Statistic: {stat:.4f}")
    print(f"p-value       : {pvalue:.4f}")

    if pvalue < alpha:
        print("➡ Conclusão: Série NÃO estacionária (rejeita H0 de estacionariedade)\n")
    else:
        print("➡ Conclusão: Série ESTACIONÁRIA (não rejeita H0)\n")

def pp_test(series, alpha=0.05):

```

```

series = series.dropna()

pp = PhillipsPerron(series, trend="c")

stat = pp.stat
pvalue = pp.pvalue

print("=== Phillips-Perron (ARCH) ===")
print(f"PP Statistic : {stat:.4f}")
print(f"p-value      : {pvalue:.4f}")

if pvalue < alpha:
    print("➡ Conclusão: Série ESTACIONÁRIA (rejeita H0 de raiz unitária)\n")
else:
    print("➡ Conclusão: Série NÃO estacionária (não rejeita H0)\n")

# Função agregadora (recomendado)
def stationarity_tests(series, alpha=0.05):
    print("\n=====")
    print("TESTES DE ESTACIONARIEDADE ")
    print("=====\\n")

    adf_test(series, alpha)
    kpss_test(series, alpha)
    pp_test(series, alpha)

```

In [13]: stationarity_tests(df_eda["traffic_volume"])

```

=====
TESTES DE ESTACIONARIEDADE
=====

=== ADF (Augmented Dickey-Fuller) ===
ADF Statistic : -28.0144
p-value      : 0.0000
➡ Conclusão: Série ESTACIONÁRIA (rejeita H0 de raiz unitária)

=== KPSS ===
KPSS Statistic: 0.2081
p-value      : 0.1000
➡ Conclusão: Série ESTACIONÁRIA (não rejeita H0)

=== Phillips-Perron (ARCH) ===
PP Statistic : -31.1963
p-value      : 0.0000
➡ Conclusão: Série ESTACIONÁRIA (rejeita H0 de raiz unitária)

```

- ACF: . Lag 1 extremamente alto (autocorrelação imediata fortíssima); . Oscilações com períodos muito claros; . Picos positivos nos lags ~24, 48, 72, 96, 120, 144, 168. . Indica sazonalidade diária e sazonalidade semanal (lag 168); . O decaimento lento é um indicador de memória longa e do aspecto sazonal. - PACF: . Lag 1 muito forte: dependência imediata direta (AR(1) component); . Lag 2 fortemente negativo: comportamento compensatório comum em séries com ciclos diários; . Após lag 3–5, a PACF praticamente zera.

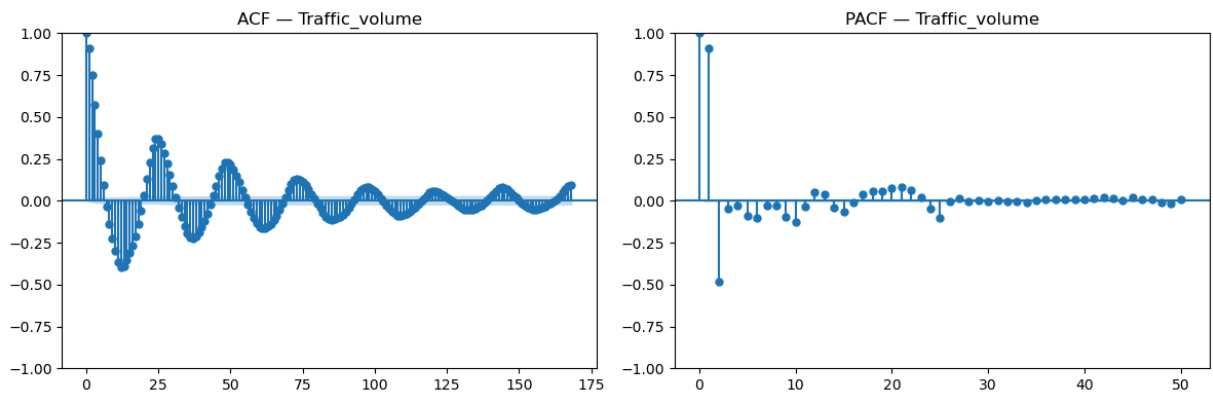
In [14]: # Ciclo diario: lags = 24
Ciclo semanal: lags = 24 * 7 = 168

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# ACF
plot_acf(df_eda["traffic_volume"], lags=168,
         ax=axes[0])
axes[0].set_title("ACF - Traffic_volume")

# PACF
plot_pacf(df_eda["traffic_volume"], lags=50,
         ax=axes[1])
axes[1].set_title("PACF - Traffic_volume")

plt.tight_layout()
plt.show()
```



- Descrição do dataset temporário

```
In [15]: df_eda[cols_num + [target]].describe().round(4)
```

```
Out[15]:
```

	temp	rain_1h	snow_1h	clouds_all	hour_sin	hour_cos	weekday_sin
count	48187.0000	48187.0000	48187.0000	48187.0000	48187.0000	48187.0000	48187.0000
mean	281.2050	0.3344	0.0002	49.3655	0.0144	0.0067	0.0010
std	13.3387	44.7970	0.0082	39.0152	0.7084	0.7056	0.7060
min	0.0000	0.0000	0.0000	0.0000	-1.0000	-1.0000	-0.9740
25%	272.1600	0.0000	0.0000	1.0000	-0.7071	-0.7071	-0.7810
50%	282.4500	0.0000	0.0000	64.0000	0.0000	0.0000	0.0000
75%	291.8060	0.0000	0.0000	90.0000	0.7071	0.7071	0.7810
max	310.0700	9831.3000	0.5100	100.0000	1.0000	1.0000	0.9740

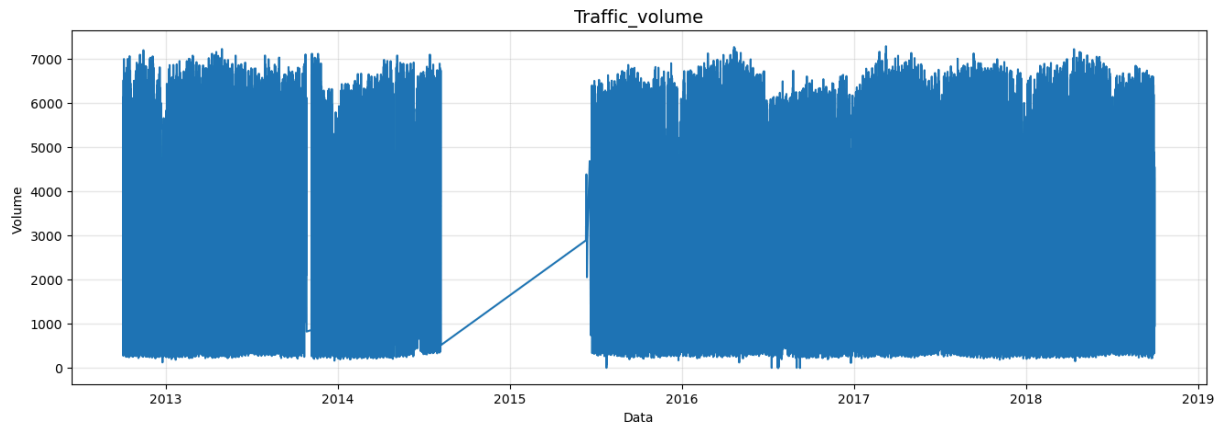
- Volume de tráfico no tempo

```
In [16]: plt.figure(figsize=(16,5))
sns.lineplot(
    data=df_eda,
```

```

x="date_time",
y="traffic_volume",
estimator=None,
errorbar=None
)
plt.title("Traffic_volume", fontsize=14)
plt.xlabel("Data")
plt.ylabel("Volume ")
plt.grid(alpha=0.3)
plt.show()

```



- Tráfico médio por hora do dia

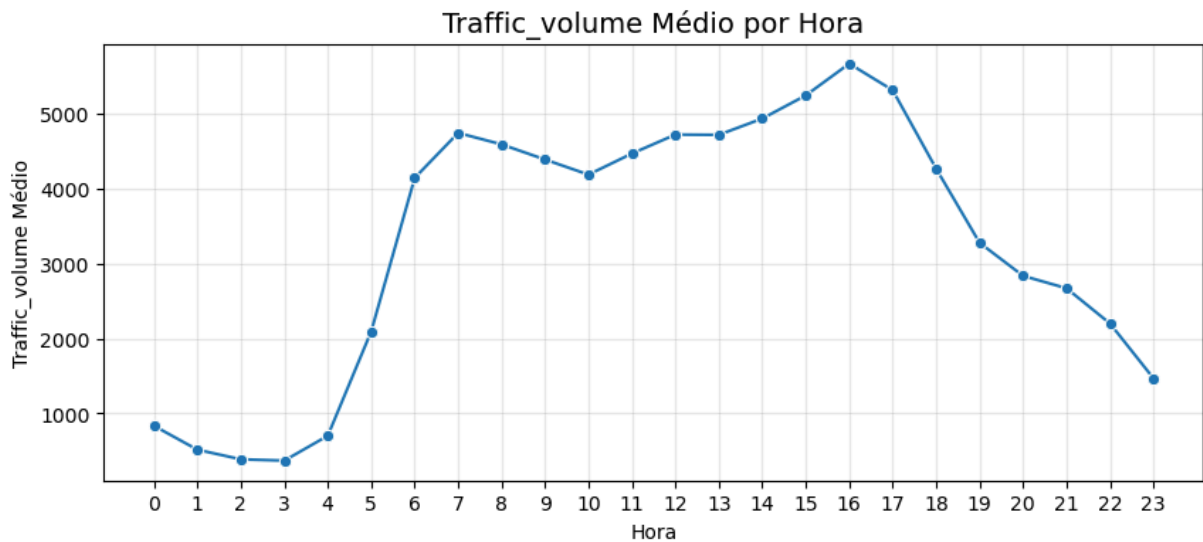
```

In [17]: plt.figure(figsize=(10,4))

sns.lineplot(
    data=df_eda,
    x="hour",
    y="traffic_volume",
    estimator="mean",    # calcula média automaticamente
    errorbar=None,      # remove intervalos de confiança
    marker="o"
)

plt.title("Traffic_volume Médio por Hora", fontsize=14)
plt.xlabel("Hora")
plt.ylabel("Traffic_volume Médio")
plt.grid(alpha=0.3)
plt.xticks(range(0, 24))
plt.show()

```

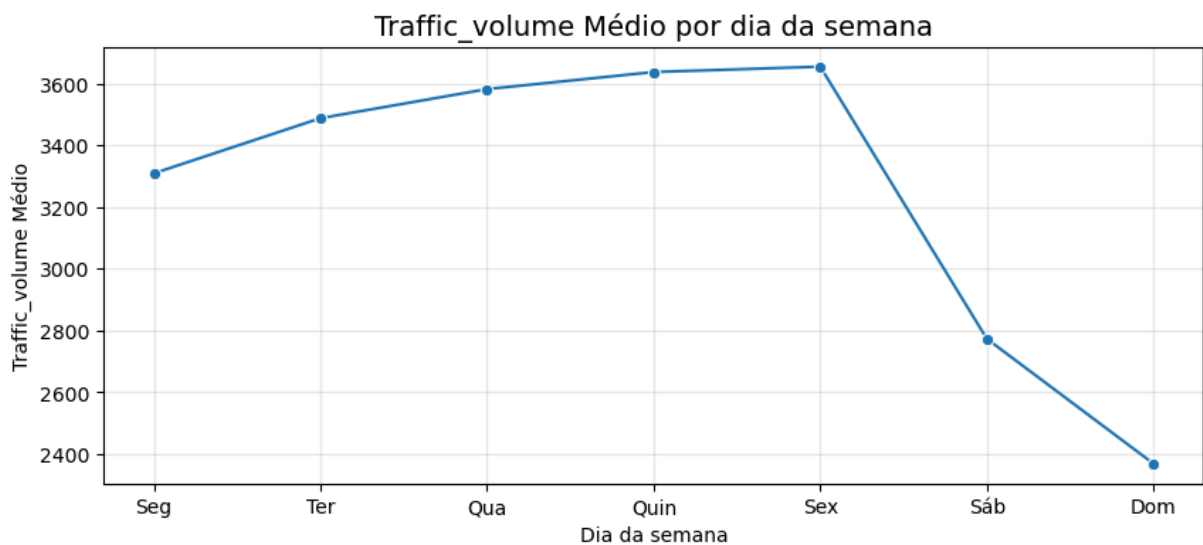



- Tráfico médio por dia da semana

```
In [18]: plt.figure(figsize=(10,4))

sns.lineplot(
    data=df_eda,
    x="weekday",
    y="traffic_volume",
    estimator="mean",
    errorbar=None,
    marker="o"
)

plt.title("Traffic_volume Médio por dia da semana", fontsize=14)
plt.xlabel("Dia da semana")
plt.ylabel("Traffic_volume Médio")
plt.grid(alpha=0.3)
plt.xticks(range(0, 7), ["Seg", "Ter", "Qua", "Quin", "Sex", "Sáb", "Dom"])
plt.show()
```



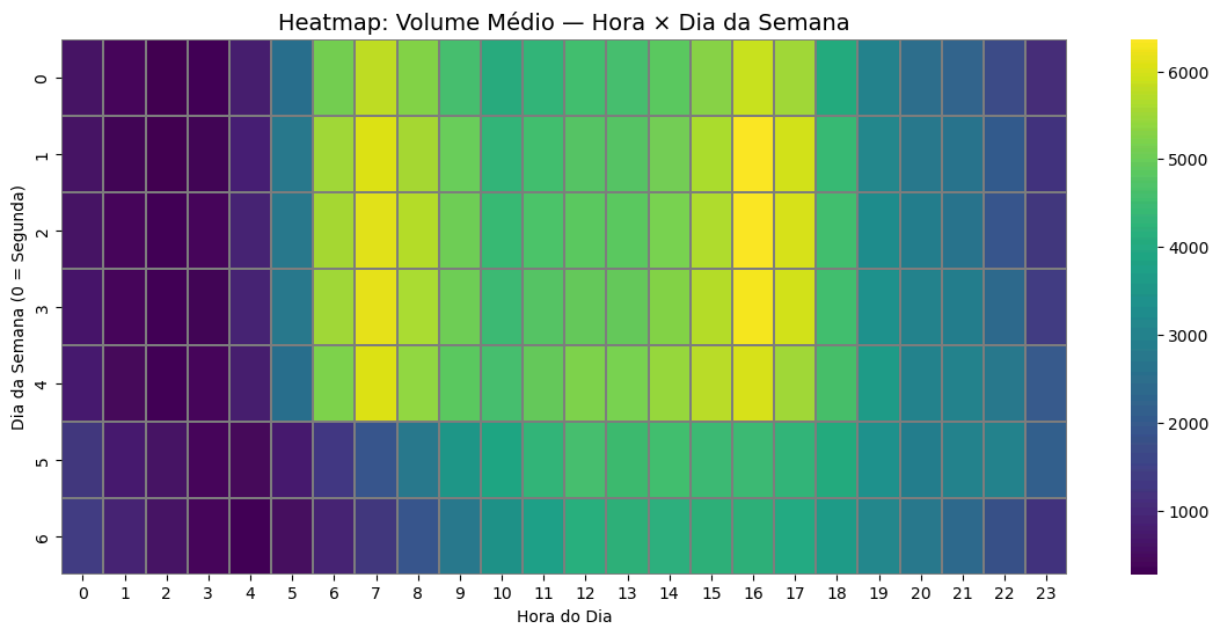
- Heatmap hora × dia da semana

- Os horários de maior volume médio ficam concentrados as 7h e as 16h.

```
In [19]: # Tabela de médias por weekday e hora
pivot = df_eda.pivot_table(
    values="traffic_volume",
    index="weekday",
    columns="hour",
    aggfunc="mean"
)

plt.figure(figsize=(14,6))
sns.heatmap(
    pivot,
    cmap="viridis",
    linewidths=0.1,
    linecolor="gray"
)

plt.title("Heatmap: Volume Médio – Hora × Dia da Semana", fontsize=14)
plt.xlabel("Hora do Dia")
plt.ylabel("Dia da Semana (0 = Segunda)")
plt.show()
```

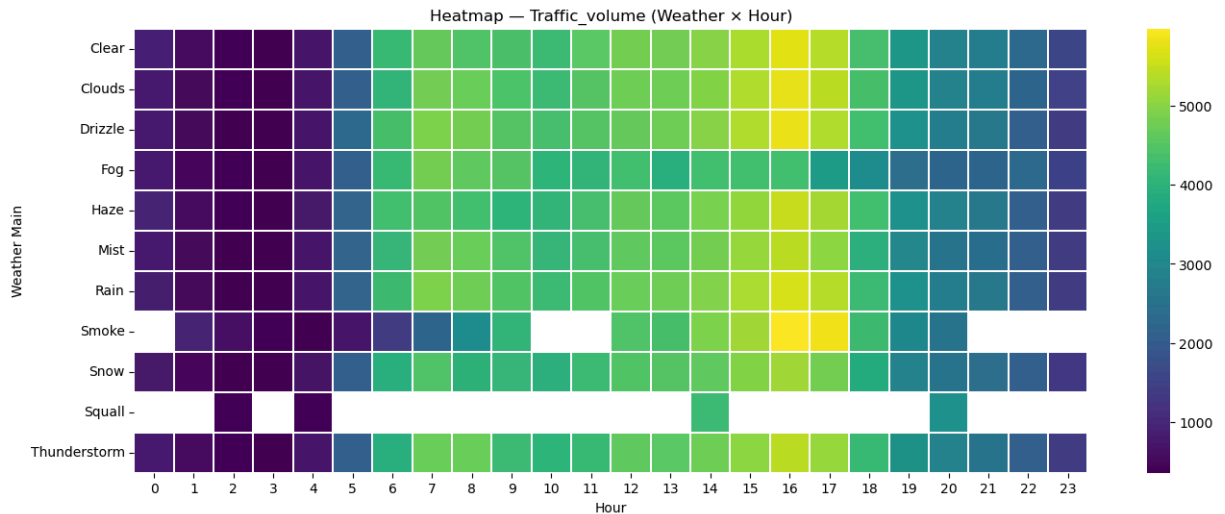


- Heatmap hora × clima

- Em dias claros, com nuvens e garoa, o horário de 16h apresenta maior tráfego.

```
In [20]: pivot = df_eda.pivot_table(
    values="traffic_volume",
    index="weather_main",
    columns="hour",
    aggfunc="mean"
)
```

```
plt.figure(figsize=(16,6))
sns.heatmap(pivot, cmap="viridis", linewidths=0.1)
plt.title("Heatmap — Traffic_volume (Weather × Hour)")
plt.xlabel("Hour")
plt.ylabel("Weather Main")
plt.show()
```



- Distribuição da variável target

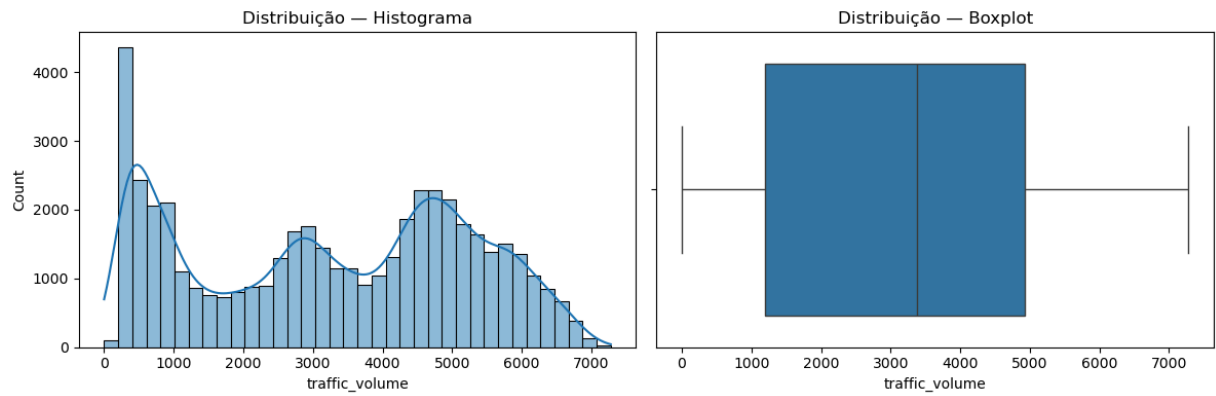
- A variável apresenta distribuição multimodal, com picos em 500, 3000 e 4500 (aproximadamente). Adicionalmente, não há outliers.

```
In [21]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Histograma
sns.histplot(
    data=df_eda,
    x="traffic_volume",
    kde=True,
    edgecolor="black",
    ax=axes[0]
)
axes[0].set_title("Distribuição — Histograma")

# Boxplot
sns.boxplot(
    data=df_eda,
    x="traffic_volume",
    ax=axes[1]
)
axes[1].set_title("Distribuição — Boxplot")

plt.tight_layout()
plt.show()
```



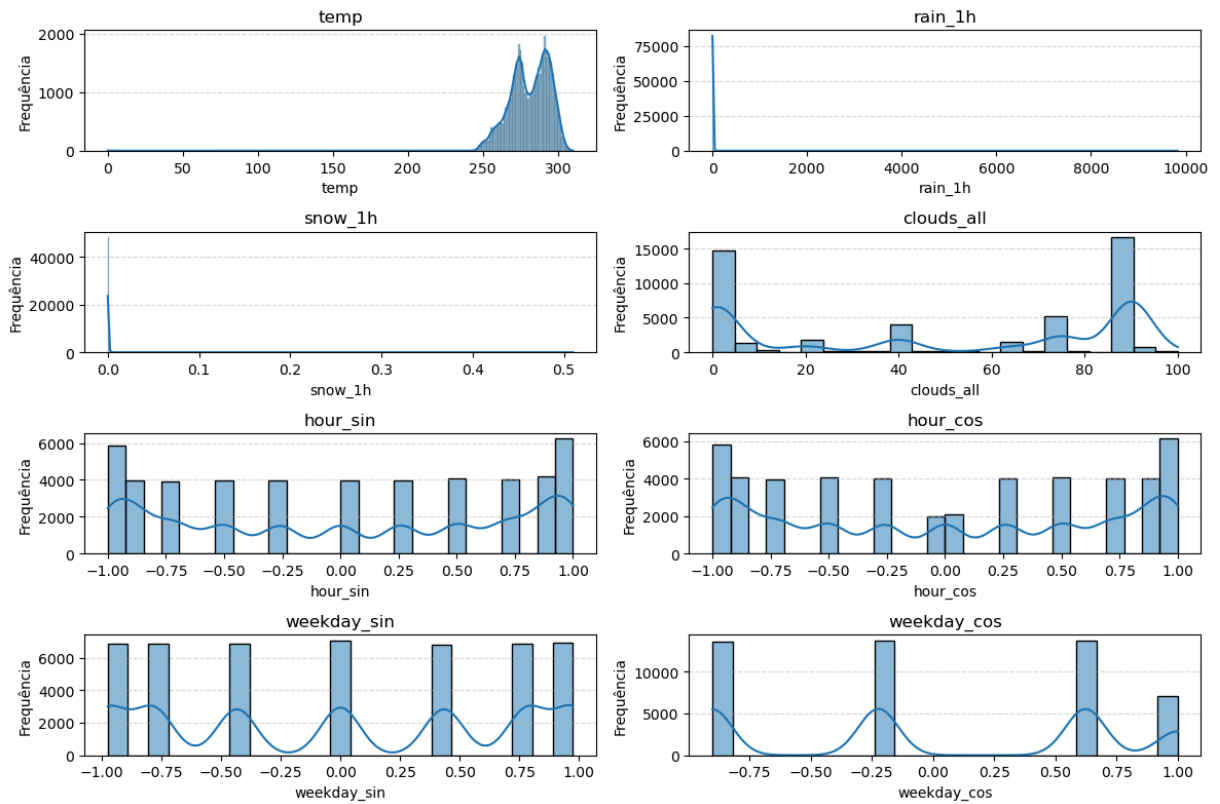
- Distribuição das variáveis numéricas

```
In [22]: fig, axes = plt.subplots(4, 2, figsize=(12, 8))
axes = axes.flatten()

for ax, col in zip(axes, cols_num):
    sns.histplot(
        data=df_eda,
        x=col,
        kde=True,
        edgecolor="black",
        ax=ax
    )
    ax.set_title(col)
    ax.set_xlabel(col)
    ax.set_ylabel("Frequência")
    ax.grid(axis="y", linestyle="--", alpha=0.5)

# Caso tenha mais eixos do que colunas numéricas
for ax in axes[len(cols_num):]:
    fig.delaxes(ax)

plt.tight_layout()
plt.show()
```

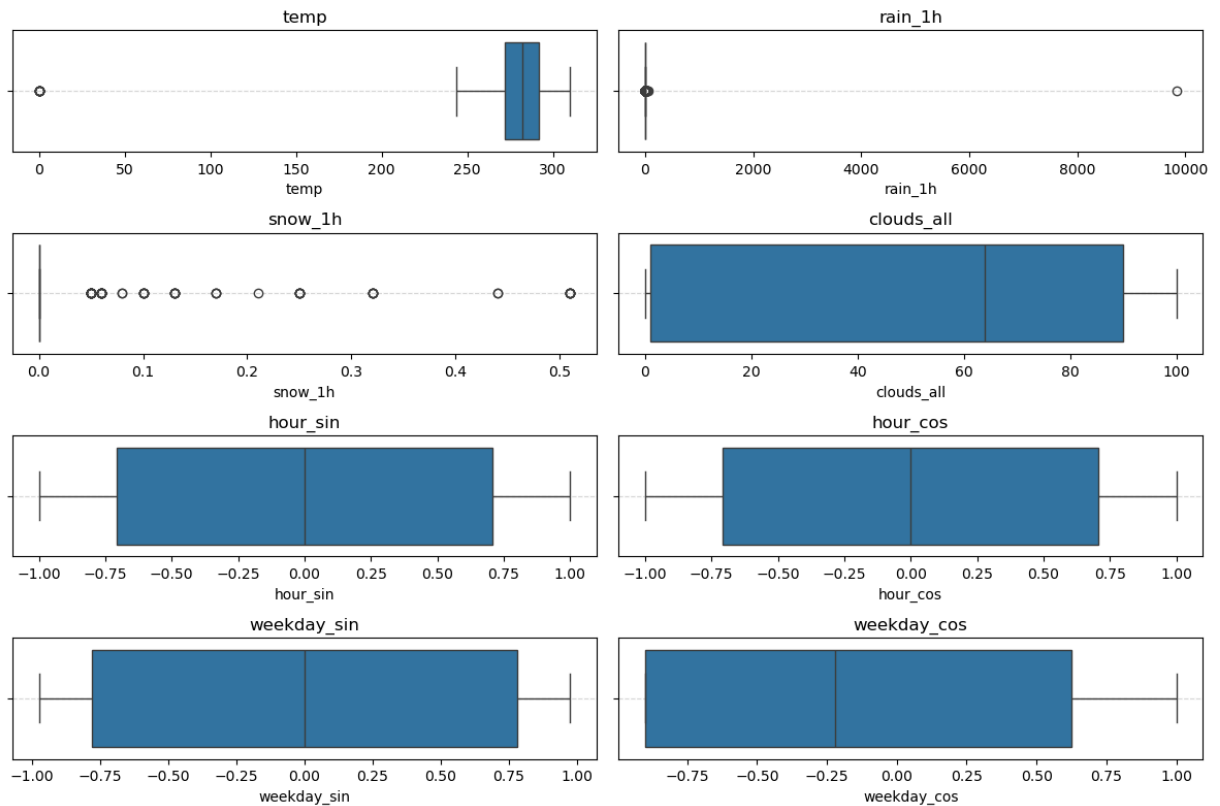


```
In [23]: fig, axes = plt.subplots(4, 2, figsize=(12, 8))
axes = axes.flatten()

for ax, col in zip(axes, cols_num):
    sns.boxplot(
        data=df_eda,
        x=col,
        ax=ax
    )
    ax.set_title(col)
    ax.grid(axis="y", linestyle="--", alpha=0.5)

# Caso tenha mais axes do que colunas numéricas
for ax in axes[len(cols_num):]:
    fig.delaxes(ax)

plt.tight_layout()
plt.show()
```



- Distribuição da variável binária

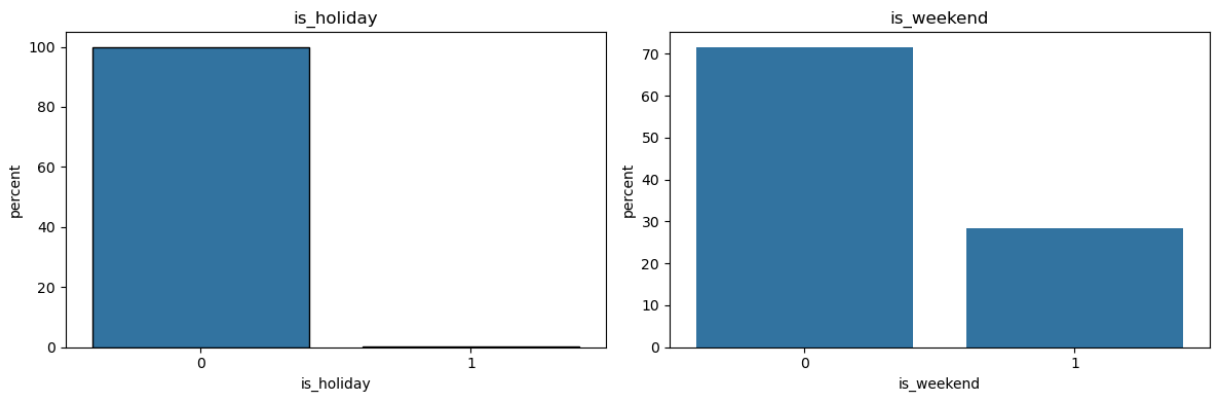
- Note como a variável is_holiday é extremamente desbalanceada.

```
In [24]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

```
# Barplot
sns.countplot(
    data=df_eda,
    x="is_holiday",
    stat="percent",
    ax=axes[0],
    edgecolor="black"
)
axes[0].set_title("is_holiday")

# Barplot
sns.countplot(
    data=df_eda,
    stat="percent",
    x="is_weekend",
    ax=axes[1]
)
axes[1].set_title("is_weekend")

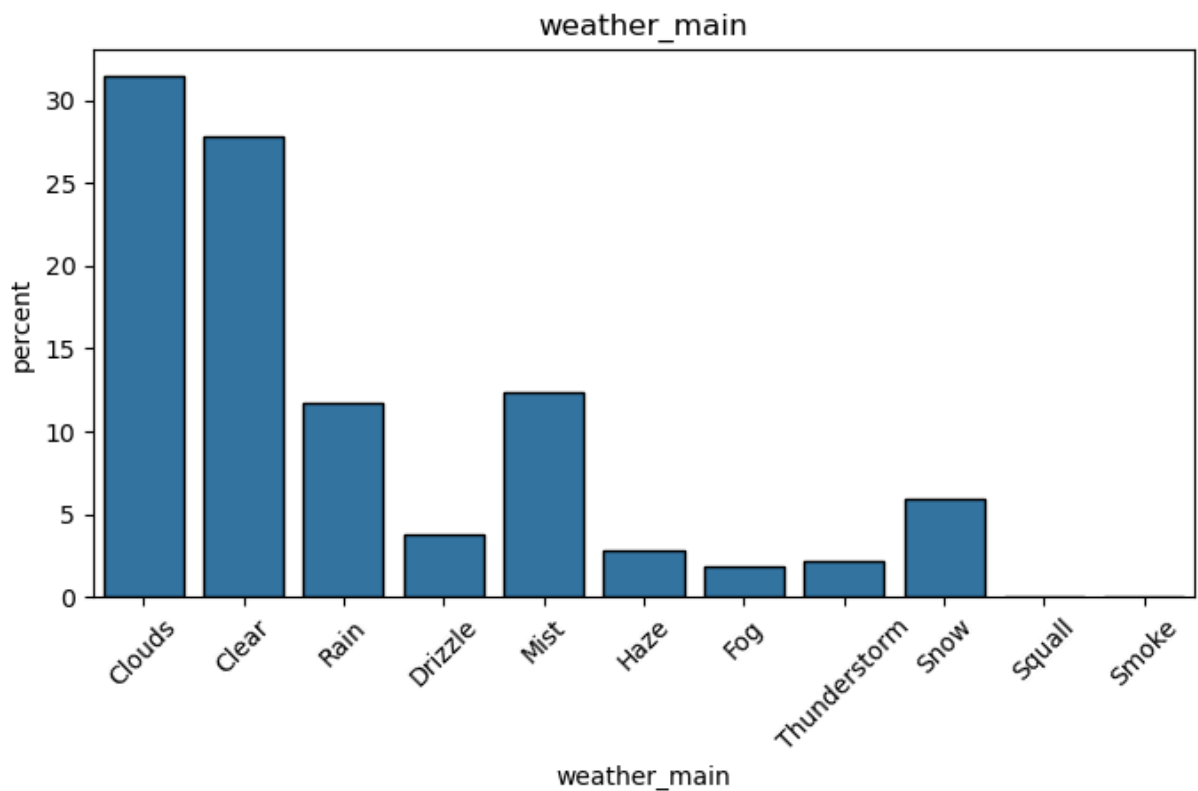
plt.tight_layout()
plt.show()
```



- Distribuição das variáveis categóricas

```
In [25]: # Weather description tem alta cardinalidade -> não plote junto com weather_main
fig, axes = plt.subplots(1, 1, figsize=(8, 4))

# Contagem (barras verticais)
sns.countplot(
    data=df_eda,
    x="weather_main",
    stat="percent",
    ax=axes,
    edgecolor="black"
)
axes.set_title("weather_main")
axes.tick_params(axis='x', rotation=45)
```



- Relação da variável target com as demais numéricas

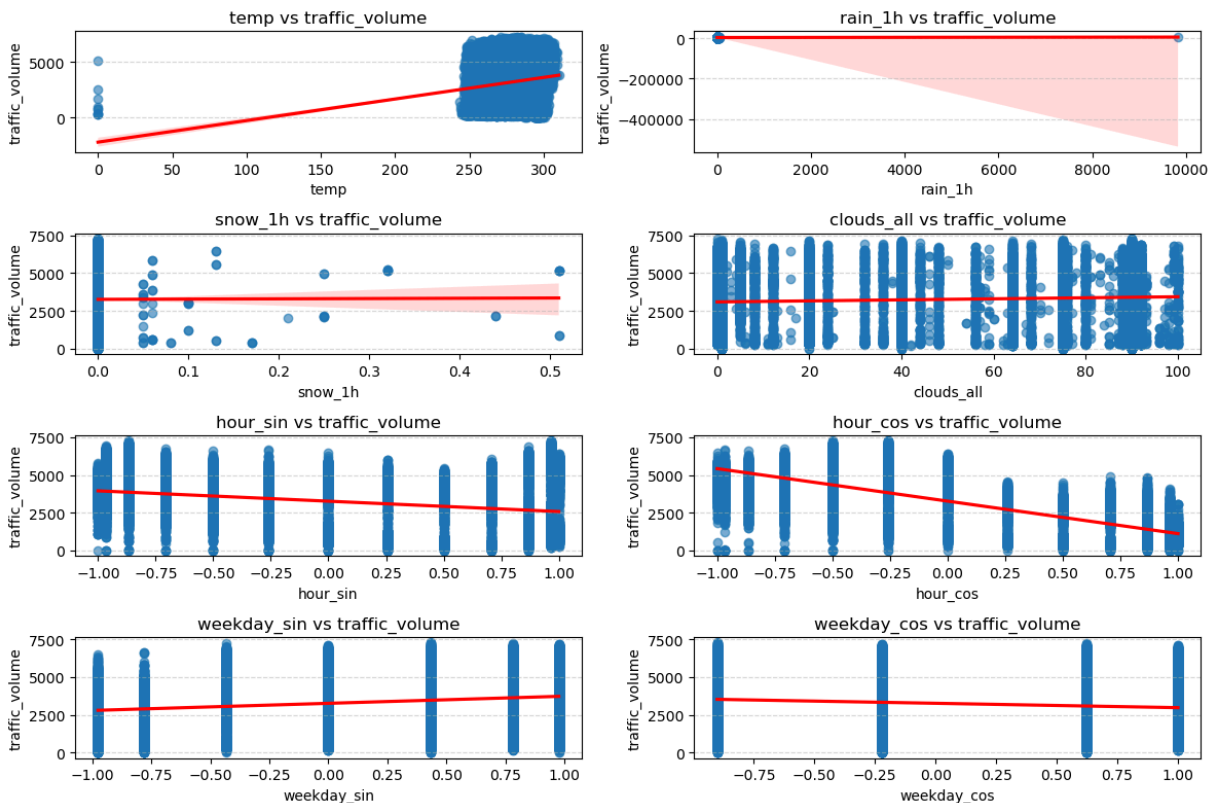
- Observa-se que, a partir da regressão linear, muitas das relações são quase horizontais, o que denota ausência de relação ou que a relação é mais complexa.

```
In [26]: fig, axes = plt.subplots(4, 2, figsize=(12, 8))
axes = axes.flatten()

for ax, col in zip(axes, cols_num):
    sns.regplot(
        data=df_eda,
        x=col,
        y=target,
        ax=ax,
        scatter_kws={"alpha": 0.6}, # pontos mais suaves
        line_kws={"color": "red"}   # linha de regressão
    )
    ax.set_title(f"{col} vs {target}")
    ax.set_xlabel(col)
    ax.set_ylabel(target)
    ax.grid(axis="y", linestyle="--", alpha=0.5)

# Remover eixos excedentes
for ax in axes[len(cols_num):]:
    fig.delaxes(ax)

plt.tight_layout()
plt.show()
```



- Relação da variável target com as variáveis binárias


```
In [27]: fig, axes = plt.subplots(2, 2, figsize=(12, 5))
```

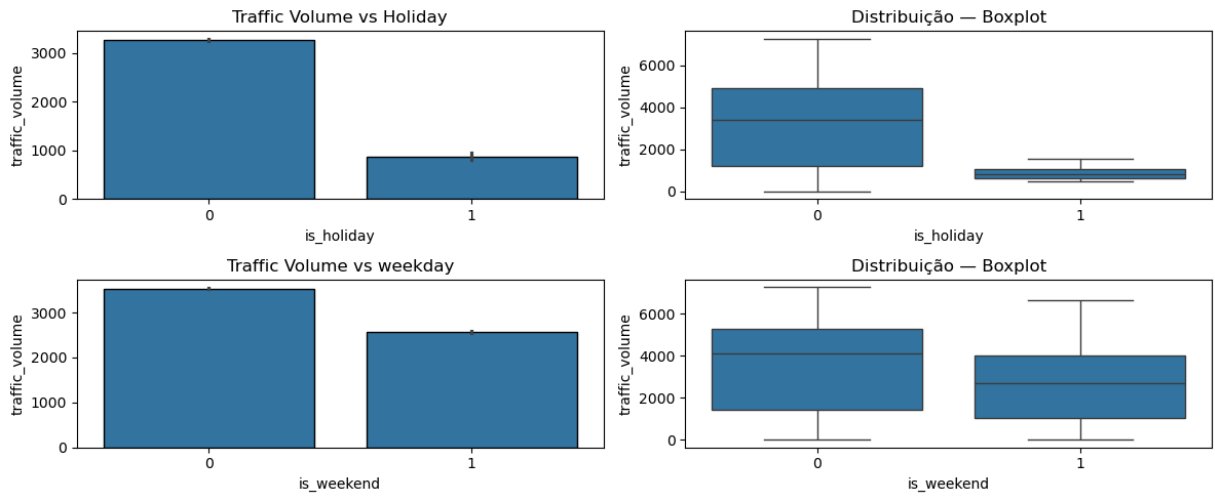
```
# Barplot
sns.barplot(
    data=df_eda,
    x="is_holiday",
    y="traffic_volume",
    edgecolor="black",
    ax=axes[0,0]
)
axes[0,0].set_title("Traffic Volume vs Holiday")

# Boxplot
sns.boxplot(
    data=df_eda,
    x="is_holiday",
    y="traffic_volume",
    ax=axes[0,1]
)
axes[0,1].set_title("Distribuição - Boxplot")

# Barplot
sns.barplot(
    data=df_eda,
    x="is_weekend",
    y="traffic_volume",
    edgecolor="black",
    ax=axes[1,0]
)
axes[1,0].set_title("Traffic Volume vs weekday")

# Boxplot
sns.boxplot(
    data=df_eda,
    x="is_weekend",
    y="traffic_volume",
    ax=axes[1,1]
)
axes[1,1].set_title("Distribuição - Boxplot")

plt.tight_layout()
plt.show()
```



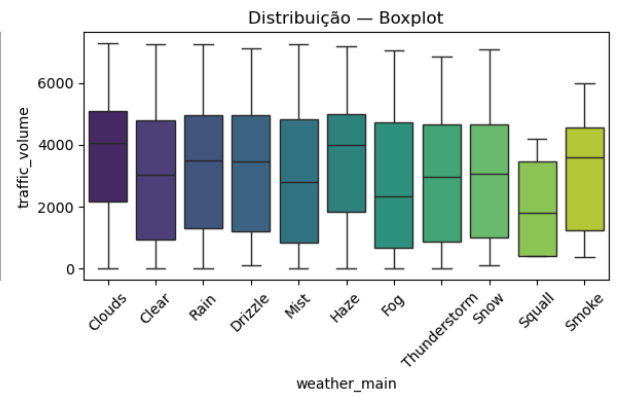
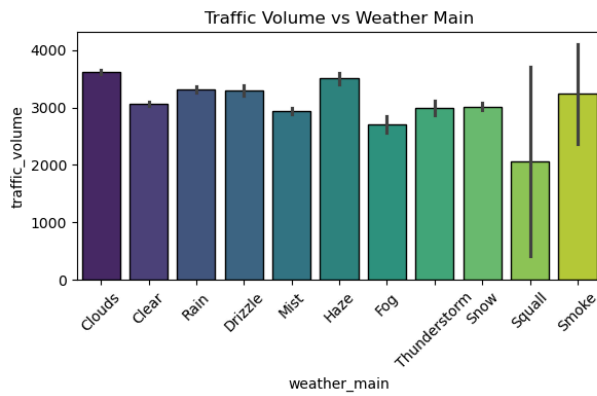
- Relação da variável target com as variáveis categóricas

```
In [28]: # Weather description tem alta cardinalidade -> não plote junto com weather_main
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Barplot
sns.barplot(
    data=df_eda,
    x="weather_main",
    y="traffic_volume",
    edgecolor="black",
    palette="viridis",
    ax=axes[0]
)
axes[0].tick_params(axis='x', rotation=45)
axes[0].set_title("Traffic Volume vs Weather Main")

# Boxplot
sns.boxplot(
    data=df_eda,
    x="weather_main",
    y="traffic_volume",
    palette="viridis",
    ax=axes[1]
)
axes[1].tick_params(axis='x', rotation=45)
axes[1].set_title("Distribuição — Boxplot")

plt.tight_layout()
plt.show()
```

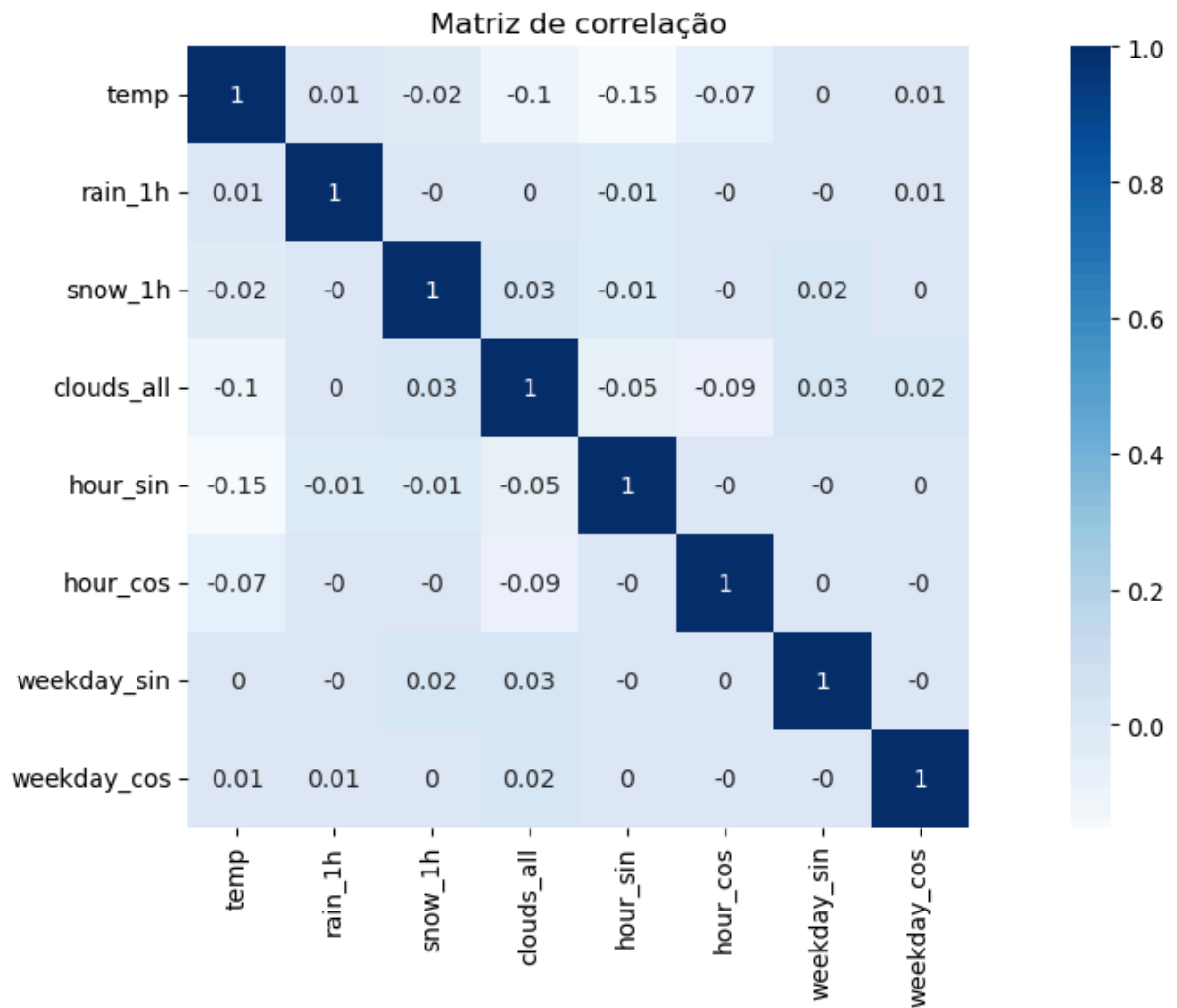


- Correlação

- A correlação linear entre as variáveis é bem fraca, o que sugere uma complexidade maior.

```
In [31]: corr=df_eda[cols_num].corr().round(2)
plt.figure(figsize=(12,6))
plt.title("Matriz de correlação")
sns.heatmap(corr, vmax=1, annot=True, square=True, cmap="Blues")

plt.tight_layout()
plt.show()
```



- Associação

Valor do V de Cramér Interpretação da Associação Comentário -----
 ----- 0.00 a 0.10 Desprezível / Nula A variável não ajuda a distinguir as classes.
 Ruído. 0.10 a 0.30 Fraca Existe um sinal leve, mas sozinha a variável não decide nada. Útil em conjunto com outras. 0.30 a 0.50 Moderada Associação Importante. Esta variável já tem poder preditivo relevante. > 0.50 Forte Determinante. A variável quase "define" o resultado. (Raro em dados sociais/humanos).- Nota-se de imediato que a associação ntre weather_description e weather_main (1), o que diz que uma quase "define" a outra.

```
In [32]: def cramers_v_corrected(confusion_matrix):
    chi2 = stats.chi2_contingency(confusion_matrix)[0]
    n = confusion_matrix.sum().sum()
    phi2 = chi2 / n
    r, k = confusion_matrix.shape

    phi2corr = max(0, phi2 - ((k - 1)*(r - 1))/(n - 1))
    rcorr = r - (r - 1)**2 / (n - 1)
    kcorr = k - (k - 1)**2 / (n - 1)

    return np.sqrt(phi2corr / min((kcorr - 1), (rcorr - 1)))

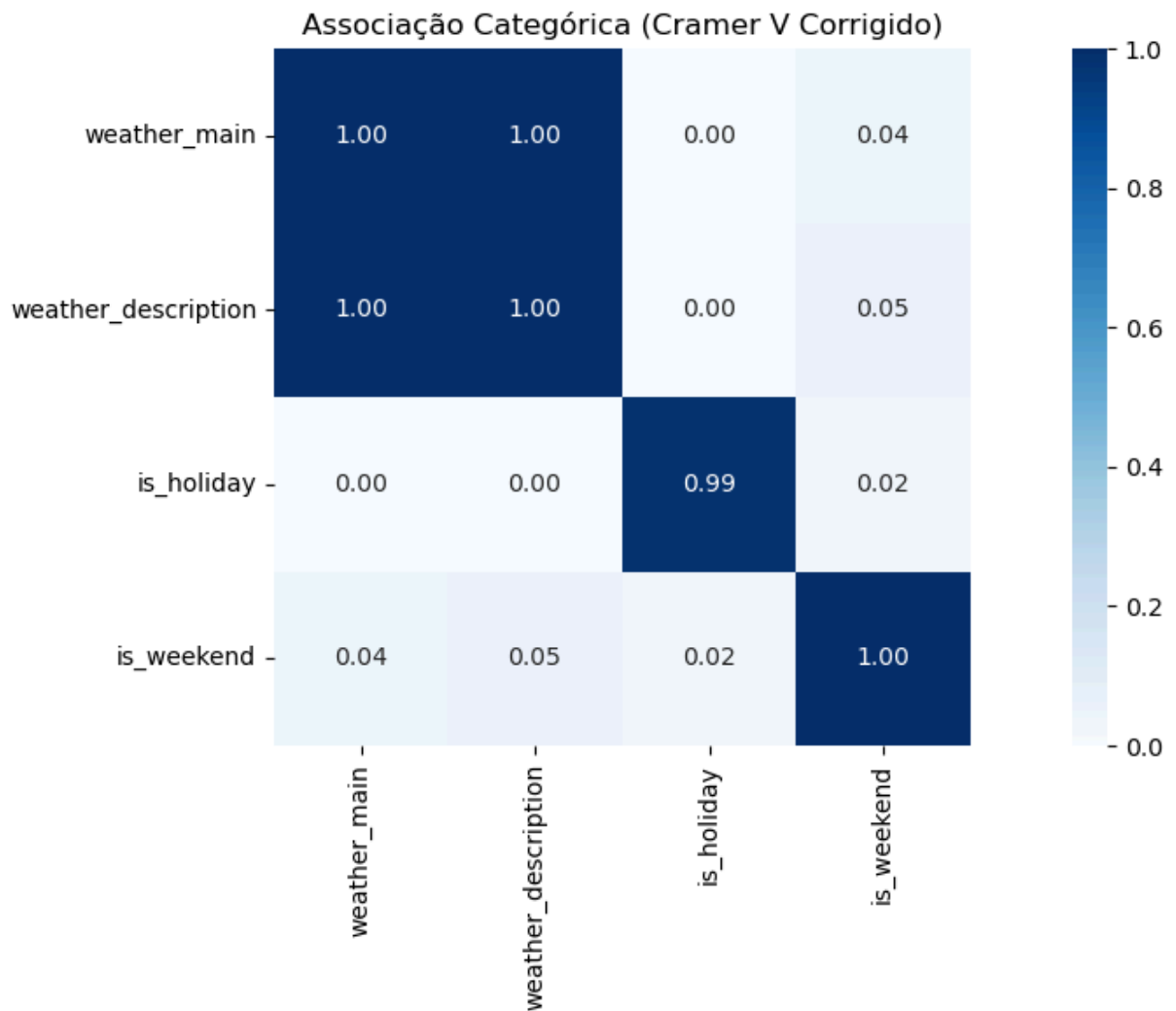
cols_v = cols_cat + cols_bin

mat = pd.DataFrame(index=cols_v, columns=cols_v)

for c1 in cols_v:
    for c2 in cols_v:
        tab = pd.crosstab(df_eda[c1], df_eda[c2])
        mat.loc[c1, c2] = cramers_v_corrected(tab.values)

plt.figure(figsize=(12,6))
sns.heatmap(mat.astype(float), annot=True, square=True,
            cmap='Blues', vmin=0, vmax=1, fmt=".2f")
plt.title("Associação Categórica (Cramer V Corrigido)")

plt.tight_layout()
plt.show();
```



- Data leakage

- Traffic_volume é autocorrelato e exige split temporal.

Pré-processamento

- Features irrelevantes

- Pelos plots de ACF e PACF, percebe-se o quanto autocorrelata a série "traffic_volume" é. Além disso, a mesma apresenta sazonalidade diária e semanal. Assim, a coluna date_time é removida. - Outra feature a ser removida é a "weather_description", já que além de ser altamente "correlacionada" (cramer's v) com "weather_main", sua alta cardinalidade pode aumentar a dimensionalidade da tarefa sem necessidade.

- Transformações nas variáveis

- Dado os insights de traffic_volume, utilizar a mesma variável defasada como feature é uma opção interessante.

- Dataset de pré-processamento e engenharia de feature

- Foram criadas variáveis de defasagem para incorporar dependências temporais de curto, médio e longo prazo. O lag de 1 hora captura a inércia imediata do tráfego, enquanto os lags de 24 e 168 horas representam, respectivamente, os ciclos diário e semanal

presentes na série temporal.

```
In [33]: # Sempre começar com o df original, não o df_eda (EDA)
df_prep = df_eda.copy()

df_prep["traffic_volume(-1)"] = df_prep["traffic_volume"].shift(1)
df_prep["traffic_volume(-24)"] = df_prep["traffic_volume"].shift(24)
df_prep["traffic_volume(-168)"] = df_prep["traffic_volume"].shift(168)

df_prep = df_prep.drop(columns=[
    "weather_description", # cardinalidade alta
    "date_time",           # substituída por hour/weekday + sin/cos
    "hour",                # substituída por sin/cos
    "weekday"              # substituída por sin/cos + is_weekend
])

df_prep = df_prep.dropna().reset_index(drop=True)
```

- Pré-processador

```
In [36]: cols_num = ["temp", "rain_1h", "snow_1h", "clouds_all",
    "hour_sin", "hour_cos", "weekday_sin", "weekday_cos",
    "traffic_volume(-1)", "traffic_volume(-24)",
    "traffic_volume(-168)"]

cols_cat = ["weather_main"]

cols_bin = ["is_holiday", "is_weekend"]

target = "traffic_volume"
```

```
In [37]: preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), cols_num),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cols_cat),
        ("bin", "passthrough", cols_bin)
    ],
    remainder="drop"
)
```

- Split temporal

```
In [38]: # O passado prevê o futuro, nunca o contrário
# O teste sempre deve ser o bloco final do dataset
# Dataset com 5,5 anos de dados diários; para este dataset (grande) um split 50/50
# Separa em ciclos diários, semanais e até anuais com esta divisão.
train_size = len(df_prep) - 20000

train = df_prep.iloc[:train_size]
test = df_prep.iloc[train_size:]

X_train = train.drop(columns=[target])
y_train = train[target]
```

```
X_test = test.drop(columns=[target])
y_test = test[target]
```

Treinamento e Avaliação de Modelos Base

- Criar um avaliador universal

```
In [39]: def evaluate_model(name, model, X_test, y_test):

    # Predição
    y_pred = model.predict(X_test)

    # Métricas
    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mape = mean_absolute_percentage_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    return {
        "Modelo": name,
        "MAE": mae,
        "RMSE": rmse,
        "MAPE": mape,
        "R2": r2
    }
```

- Pipeline para cada modelo

```
In [40]: pipelines = {
    "Linear Regression": Pipeline([
        ("preprocess", preprocessor),
        ("model", LinearRegression())
    ]),

    "Decision Tree": Pipeline([
        ("preprocess", preprocessor),
        ("model", DecisionTreeRegressor(
            random_state=SEED
        ))
    ]),

    "Random Forest": Pipeline([
        ("preprocess", preprocessor),
        ("model", RandomForestRegressor(
            random_state=SEED
        ))
    ]),

    "XGBoost": Pipeline([
        ("preprocess", preprocessor),
```

```

        ("model", XGBRegressor(
            objective="reg:squarederror",
            random_state=SEED
        ))
    ],

    "LGBM": Pipeline([
        ("preprocess", preprocessor),
        ("model", LGBMRegressor(
            random_state=SEED,
            verbosity=-1
        ))
    ])

}

results = []

for name, pipe in pipelines.items():
    pipe.fit(X_train, y_train)
    metrics = evaluate_model(name, pipe, X_test, y_test)
    results.append(metrics)

```

```

File "C:\Users\rodri\anaconda3\Lib\site-packages\joblib\externals\loky\backend\con
text.py", line 257, in _count_physical_cores
    cpu_info = subprocess.run(
        "wmic CPU Get NumberOfCores /Format:csv".split(),
        capture_output=True,
        text=True,
    )
File "C:\Users\rodri\anaconda3\Lib\subprocess.py", line 554, in run
    with Popen(*popenargs, **kwargs) as process:
        ~~~~~^~~~~~
File "C:\Users\rodri\anaconda3\Lib\subprocess.py", line 1039, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
    ~~~~~^~~~~~
        pass_fds, cwd, env,
        ~~~~~^~~~~~
    ...<5 lines>...
        gid, gids, uid, umask,
        ~~~~~^~~~~~
        start_new_session, process_group)
        ~~~~~^~~~~~
File "C:\Users\rodri\anaconda3\Lib\subprocess.py", line 1554, in _execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
    ~~~~~^~~~~~
        # no special security
        ~~~~~^~~~~~
    ...<4 lines>...
        cwd,
        ~~~~~^~~~~~
        startupinfo)
        ~~~~~^~~~~~

```

- Compilar resultados


```
In [41]: df_results = pd.DataFrame(results)
df_results = df_results.sort_values("RMSE")
df_results
```

```
Out[41]:
```

	Modelo	MAE	RMSE	MAPE	R2
4	LGBM	176.164921	274.606110	0.083014	0.980640
2	Random Forest	168.298074	276.394043	0.073086	0.980387
3	XGBoost	177.635941	280.303309	0.082839	0.979828
1	Decision Tree	235.119000	401.949149	0.098695	0.958521
0	Linear Regression	453.200210	624.657407	0.267377	0.899822

- Melhor baseline: LGBM e Random Forest - Regressão linear "engana" apesar do alto R2, a tarefa em questão (traffic_volume) é consideravelmente não linear.

Modelos Promissores

- Modelos muito próximos tecnicamente: - Observações: . Menor MAE e MAPE → melhor robustez aos outliers; . Menor RMSE e maior R2 → melhor capacidade de modelagem. - LGBM vence em RMSE e R2 e o Random Forest nas demais, o modelo escolhido será o primeiro.

```
In [42]: # Em séries temporais, não é permitido embaralhar os dados (shuffle=False).
# Se você usar KFold normal, o modelo pode treinar com dados do futuro e testar com
# o que é uma violação temporal grave. Isso pode causar data Leakage!
tscv = TimeSeriesSplit(n_splits=5)

for fold, (train_idx, test_idx) in enumerate(tscv.split(df)):
    print(f"Fold {fold+1}")
    print("Train:", train_idx[:5], "...", train_idx[-5:])
    print("Test :", test_idx[:5], "...", test_idx[-5:])
    print()
```

Fold 1

Train: [0 1 2 3 4] ... [8029 8030 8031 8032 8033]

Test : [8034 8035 8036 8037 8038] ... [16063 16064 16065 16066 16067]

Fold 2

Train: [0 1 2 3 4] ... [16063 16064 16065 16066 16067]

Test : [16068 16069 16070 16071 16072] ... [24097 24098 24099 24100 24101]

Fold 3

Train: [0 1 2 3 4] ... [24097 24098 24099 24100 24101]

Test : [24102 24103 24104 24105 24106] ... [32131 32132 32133 32134 32135]

Fold 4

Train: [0 1 2 3 4] ... [32131 32132 32133 32134 32135]

Test : [32136 32137 32138 32139 32140] ... [40165 40166 40167 40168 40169]

Fold 5

Train: [0 1 2 3 4] ... [40165 40166 40167 40168 40169]

Test : [40170 40171 40172 40173 40174] ... [48199 48200 48201 48202 48203]

- LGBM

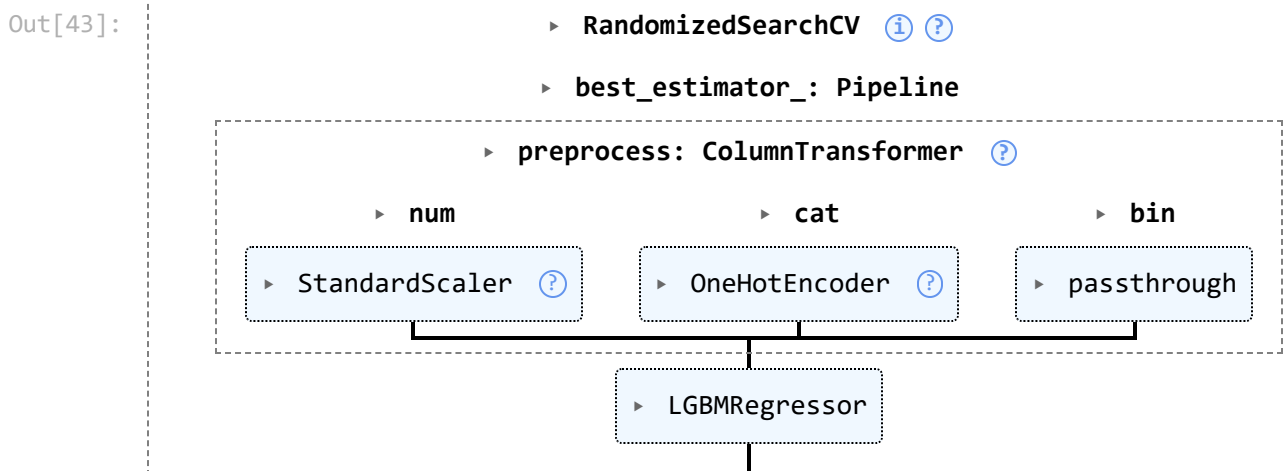
```
In [43]: lgbm_param_grid = {
    "model__n_estimators": randint(300, 800),
    "model__max_depth": randint(3, 12),
    "model__num_leaves": randint(15, 100),
    "model__learning_rate": uniform(0.01, 0.2),
    "model__subsample": uniform(0.6, 0.4),
    "model__colsample_bytree": uniform(0.6, 0.4),
    "model__min_child_samples": randint(5, 50)
}

lgbm_pipe = Pipeline([
    ("preprocess", preprocessor),
    ("model", LGBMRegressor(
        random_state=SEED,
        verbosity=-1
    ))
])

lgbm_search = RandomizedSearchCV(
    lgbm_pipe,
    param_distributions=lgbm_param_grid,
    n_iter=25,
    scoring="neg_root_mean_squared_error",
    cv=tscv,
    n_jobs=-1,
    random_state=SEED,
    verbose=2
)

lgbm_search.fit(X_train, y_train)
```

Fitting 5 folds for each of 25 candidates, totalling 125 fits



- Random forest

```
In [45]: rf_param_grid = {
    "model__n_estimators": randint(200, 600),
```

```

    "model__max_depth": randint(5, 30),
    "model__min_samples_split": randint(2, 20),
    "model__min_samples_leaf": randint(1, 10),
    "model__max_features": ["sqrt", "log2", None]
}

rf_pipe = Pipeline([
    ("preprocess", preprocessor),
    ("model", RandomForestRegressor(random_state=42))
])

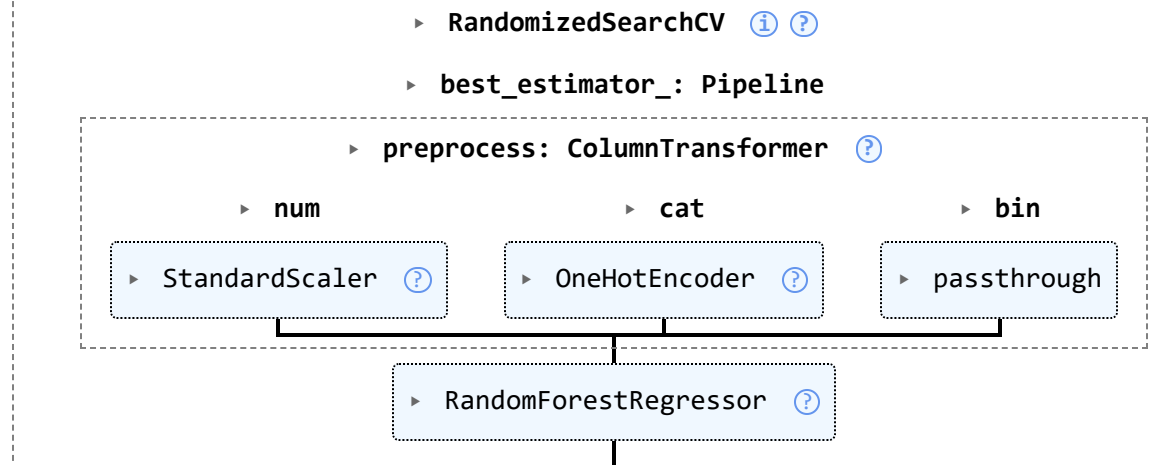
rf_search = RandomizedSearchCV(
    rf_pipe,
    param_distributions=rf_param_grid,
    n_iter=100,
    scoring="neg_root_mean_squared_error",
    cv=tscv,
    random_state=42,
    n_jobs=-1,
    verbose=2
)

rf_search.fit(X_train, y_train)

```

Fitting 5 folds for each of 100 candidates, totalling 500 fits

Out[45]:



- Comparação dos modelos promissores

```

In [46]: best_models = {
    "LGBM (Tuned)": lgbm_search.best_estimator_,
    "Random Forest (Tuned)": rf_search.best_estimator_
}

results_tuned = []

for name, model in best_models.items():
    metrics = evaluate_model(name, model, X_test, y_test)
    results_tuned.append(metrics)

df_tuned_results = pd.DataFrame(results_tuned)

```

```
df_tuned_results = df_tuned_results.sort_values("RMSE")
df_tuned_results
```

Out[46]:

	Modelo	MAE	RMSE	MAPE	R2
0	LGBM (Tuned)	168.801101	267.987910	0.077405	0.981562
1	Random Forest (Tuned)	165.356827	270.374731	0.071432	0.981232

Análise de Erros e Interpretação do Modelo Final

- Em conjunto, os resultado dos testes sugerem que o modelo é preditivamente forte, mas ainda não esgota a estrutura temporal da série, mantendo padrões residuais de autocorrelação e variância não constante, típicos de sistemas de tráfego urbano complexos.

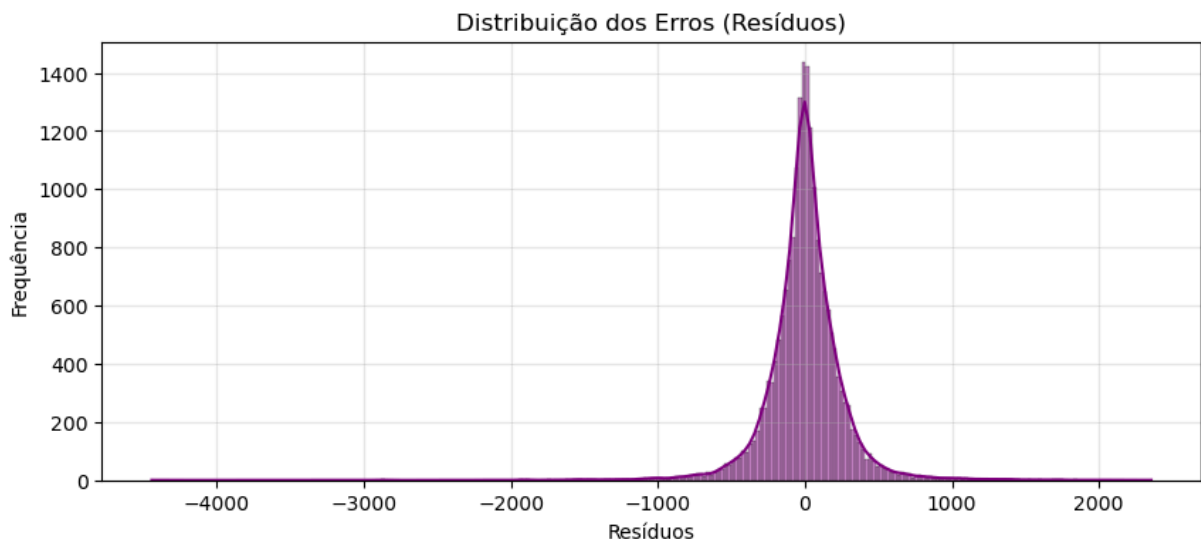
- Diagnóstico dos resíduos

```
In [47]: modelo = lgbm_search.best_estimator_

y_pred = modelo.predict(X_test)
res = y_test - y_pred

X_exog = modelo.named_steps["preprocess"].transform(X_test)

plt.figure(figsize=(10,4))
sns.histplot(res,kde=True, color="purple")
plt.title("Distribuição dos Erros (Resíduos)")
plt.xlabel("Resíduos")
plt.ylabel("Frequência")
plt.grid(alpha=0.3)
plt.show()
```



```
In [48]: alpha = 0.05

print("\n" + "="*100)
```

```

print("DIAGNÓSTICO DOS RESÍDUOS DO MODELO")
print("="*100)

# Jarque-Bera – Normalidade dos resíduos
jb_stat, jb_pvalue = jarque_bera(res)

print("\n[1] Jarque-Bera – Normalidade dos Resíduos")
print("="*100)
print(f"JB Statistic : {jb_stat:.4f}")
print(f"p-value      : {jb_pvalue:.6f}")

if jb_pvalue < alpha:
    print("Conclusão: Rejeita  $H_0$  → resíduos NÃO seguem normalidade "
          "(resultado esperado em modelos não lineares).")
else:
    print("Conclusão: Não rejeita  $H_0$  → resíduos aproximadamente normais.")

# Breusch-Pagan – Heterocedasticidade
bp_stat, bp_pvalue, f_stat, f_pvalue = het_breuschpagan(res, X_exog)

print("\n[2] Breusch-Pagan – Heterocedasticidade dos Resíduos")
print("="*100)
print(f"LM Statistic : {bp_stat:.4f}")
print(f"LM p-value   : {bp_pvalue:.6f}")
print(f"F Statistic  : {f_stat:.4f}")
print(f"F p-value    : {f_pvalue:.6f}")

if bp_pvalue < alpha:
    print("Conclusão: Rejeita  $H_0$  → evidência de heterocedasticidade "
          "(variância não constante dos resíduos).")
else:
    print("Conclusão: Não rejeita  $H_0$  → sem evidência de heterocedasticidade.")

# Ljung-Box – Autocorrelação dos resíduos

lags_test = [24, 168]
ljung = acorr_ljungbox(res, lags=lags_test, return_df=True)

print("\n[3] Ljung-Box – Autocorrelação dos Resíduos")
print("="*100)
print(ljung)

for lag in lags_test:
    p_val = ljung.loc[lag, "lb_pvalue"]
    if p_val < alpha:
        print(f"Conclusão (lag {lag:>3}): Rejeita  $H_0$  → autocorrelação residual "
              f"presente (ciclo remanescente).")
    else:
        print(f"Conclusão (lag {lag:>3}): Não rejeita  $H_0$  → sem autocorrelação "
              f"significativa.")

print("\n" + "="*100)

```

```
print("FIM DO DIAGNÓSTICO DOS RESÍDUOS")
print("=*100)
```

DIAGNÓSTICO DOS RESÍDUOS DO MODELO

[1] Jarque-Bera – Normalidade dos Resíduos

JB Statistic : 272850.4116

p-value : 0.000000

Conclusão: Rejeita H_0 → resíduos NÃO seguem normalidade (resultado esperado em modelos não lineares).

[2] Breusch-Pagan – Heterocedasticidade dos Resíduos

LM Statistic : 59.1616

LM p-value : 0.000050

F Statistic : 2.8225

F p-value : 0.000017

Conclusão: Rejeita H_0 → evidência de heterocedasticidade (variância não constante dos resíduos).

[3] Ljung-Box – Autocorrelação dos Resíduos

	lb_stat	lb_pvalue
24	7358.580277	0.0
168	9849.968964	0.0

Conclusão (lag 24): Rejeita H_0 → autocorrelação residual presente (ciclo remanescente).

Conclusão (lag 168): Rejeita H_0 → autocorrelação residual presente (ciclo remanescente).

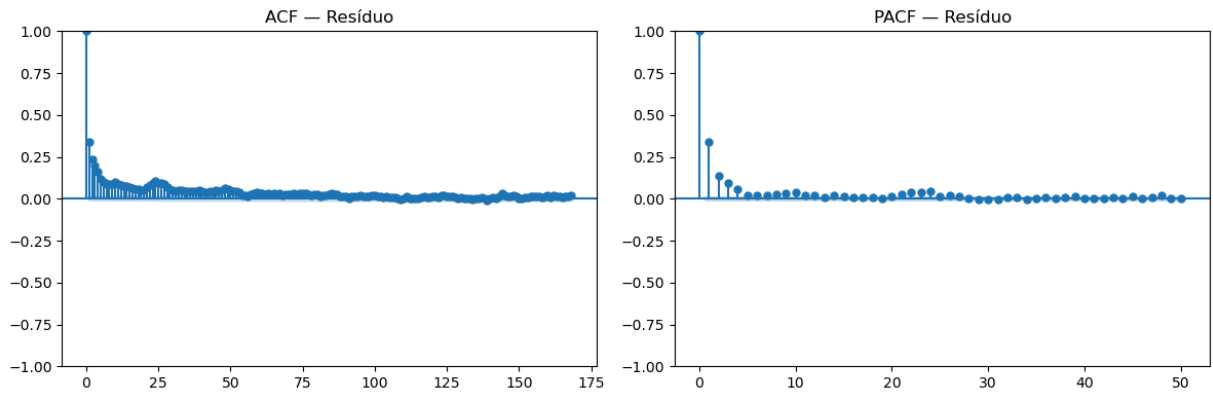
FIM DO DIAGNÓSTICO DOS RESÍDUOS

In [49]: `fig, axes = plt.subplots(1, 2, figsize=(12, 4))`

```
# ACF
plot_acf(res, lags=168,
          ax=axes[0])
axes[0].set_title("ACF – Resíduo")

# PACF
plot_pacf(res, lags=50,
           ax=axes[1])
axes[1].set_title("PACF – Resíduo")
```

```
plt.tight_layout()
plt.show()
```



- Shap Summary

```
In [ ]: - O volume de tráfego presente é fortemente explicado pelo seu volume de uma hora a
```

```
In [51]: # Dados originais
X_raw = X_test.copy()

# Dados transformados
X_transformed = modelo.named_steps["preprocess"].transform(X_test)

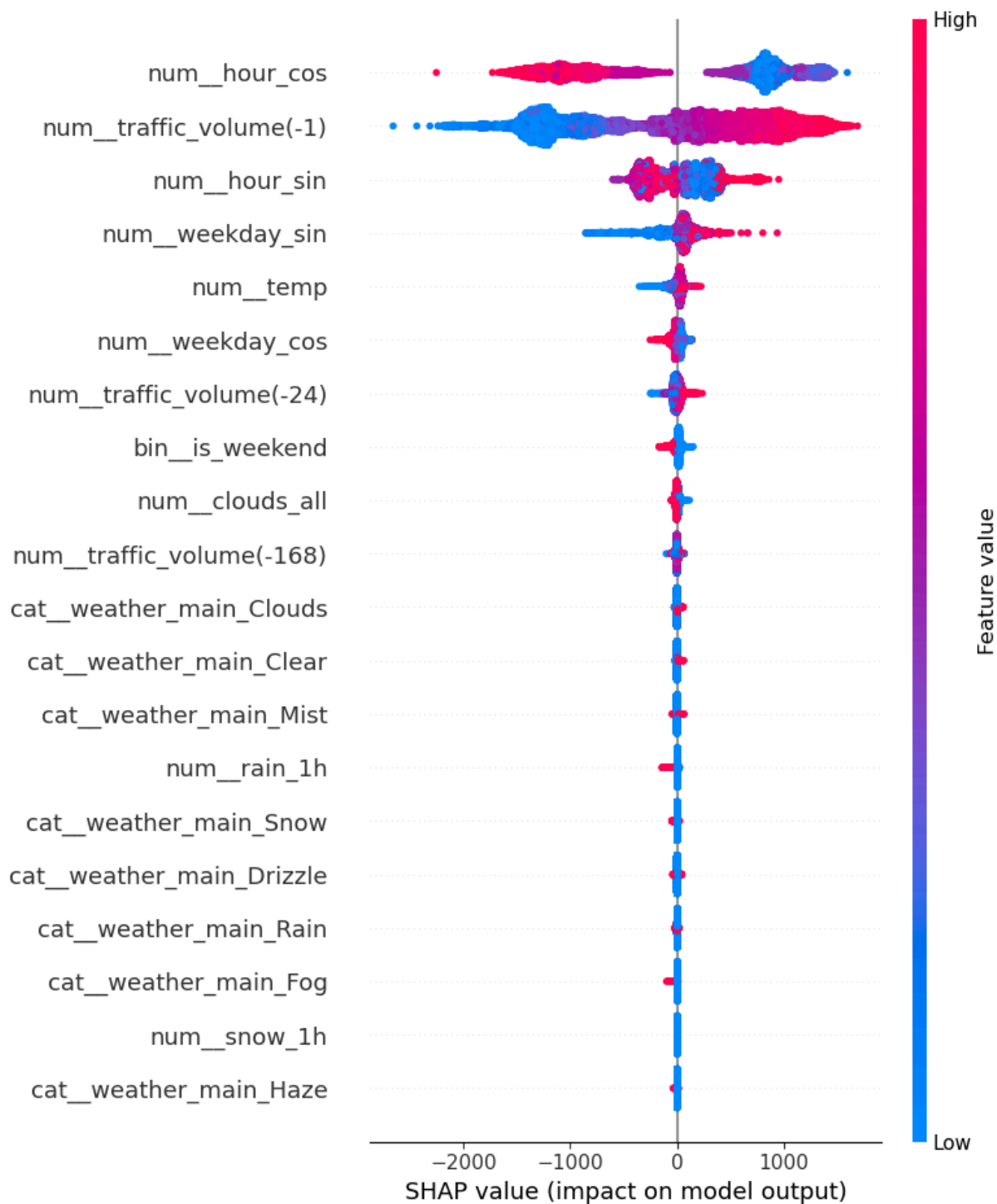
feature_names = modelo.named_steps["preprocess"].get_feature_names_out()

model = modelo.named_steps["model"]

explainer = shap.TreeExplainer(model)

# Calcular SHAP values no espaço transformado
shap_values = explainer.shap_values(X_transformed)

shap.summary_plot(
    shap_values,
    X_transformed,
    feature_names=feature_names
)
```



- Shap force plot

- A diferença entre o valor esperado e o valor efetivo é igual a aproximadamente 1638 - Partindo do valor médio (3244), a variável `num_hour_cos` representa aproximadamente 55% da variação total até a previsão final (4882).

```
In [58]: # Foi escolhido um top 5 ao invés da decomposição completa, por motivos de poluição
i = 0
TOP_K = 5

shap_i = shap_values[i]
x_i = X_transformed[i]
```



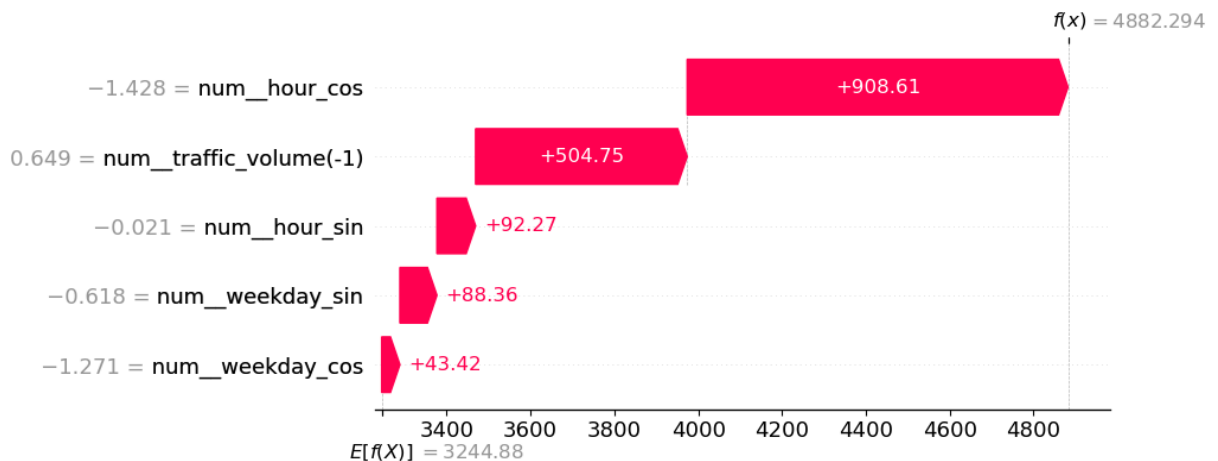
```

top_idx = np.argsort(np.abs(shap_i))[-TOP_K:]

shap_i_top = shap_i[top_idx]
x_i_top = x_i[top_idx]
feature_names_top = np.array(feature_names)[top_idx]

shap.waterfall_plot(
    shap.Explanation(
        values=shap_i_top,
        base_values=explainer.expected_value,
        data=x_i_top,
        feature_names=feature_names_top
    )
)

```



Salvar Artefatos

```

In [67]: def save_and_show(fig_name):
        """
        Salva a figura atual como SVG e exibe no notebook.

        fig_name: nome do arquivo sem extensão
        """
        plt.savefig(f"{fig_name}.svg", format="svg", bbox_inches="tight")
        plt.close()

```

```

In [68]: shap.summary_plot(
        shap_values,
        X_transformed,
        feature_names=feature_names,
        show=False
    )

    save_and_show("shap_summary")

```

```

In [66]: i = 0
        TOP_K = 5

        shap_i = shap_values[i]

```

```

x_i = X_transformed[i]

top_idx = np.argsort(np.abs(shap_i))[-TOP_K:]

shap_i_top = shap_i[top_idx]
x_i_top = x_i[top_idx]
feature_names_top = np.array(feature_names)[top_idx]

# cria figura explicitamente
plt.figure()

shap.waterfall_plot(
    shap.Explanation(
        values=shap_i_top,
        base_values=explainer.expected_value,
        data=x_i_top,
        feature_names=feature_names_top
    ),
    show=False
)

plt.savefig(
    "shap_waterfall_top5.svg",
    format="svg",
    bbox_inches="tight"
)

```

Síntese

- O objetivo do projeto foi prever o volume horário de tráfego a partir de variáveis climáticas, temporais e defasagens da própria série, com foco em desempenho preditivo, robustez e interpretabilidade via SHAP. - Para isso, foi adotada uma abordagem de engenharia de features voltada para séries temporais. As variáveis de tempo foram tratadas de forma cíclica (seno e cosseno), permitindo ao modelo capturar corretamente a natureza contínua de padrões como hora, dia da semana e mês. Além disso, foram criadas variáveis de defasagem (lags de 1h, 24h e 168h), o que permitiu incorporar a dependência histórica da série, evidenciada por testes de estacionariedade e autocorrelação. Variáveis como feriados e fins de semana também foram incluídas como binárias para capturar efeitos sazonais. - Na modelagem, o LGBM apresentou o melhor desempenho geral entre os modelos testados, superando ligeiramente o Random Forest em métricas como (RMSE e R^2). Ainda assim, o alto poder explicativo observado em ambos os modelos sugere forte estrutura determinística na série, o que exige cautela quanto a possíveis efeitos de correlação entre variáveis e desenho do split temporal. - A validação foi feita de forma estritamente temporal, evitando vazamento de dados e garantindo uma avaliação mais realista do desempenho fora da amostra. - Do ponto de vista dos resultados, os SHAP values indicam que o tráfego é majoritariamente explicado pela sua própria inércia (especialmente o lag de 1 hora) e por padrões cíclicos diários, representados pelas transformações seno e cosseno da hora. - Por fim, apesar do bom desempenho, o modelo ainda apresenta autocorrelação residual, especialmente em ciclos de 24h e 168h, indicando que parte da estrutura temporal não foi completamente capturada. Além disso, a ausência de variáveis contextuais como acidentes, obras e eventos específicos limita o potencial explicativo do modelo.